

KOHLER AI Bathroom Designer & Planner

An end-to-end AI system for real-time, catalog-verified bathroom design — from natural-language and vision-based input, through multi-constraint optimization, to 3D/AR visualization and deployment.

TRACK

Track 1 — KOHLER AI Bathroom Designer & Planner

NAME

Archita Jha

PRN

1262243200

PROGRAM

T.Y. B.Tech CSE (AI-DS) Panel F

TECHNOLOGY STACK

Frontend — React 19, Vite 8, React Three Fiber, Three.js, Drei · **Backend** — FastAPI, SQLAlchemy, PostgreSQL / SQLite · **AI / LLM** — Groq (openai/gpt-oss-120b), Ollama (llama3.2), deterministic intent router · **Data** — Playwright scraper, verified KOHLER India catalog · **3D/AR** — Studio Kohler & BIMobject models, WebXR, Apple AR Quick Look · **Deployment** — Render (backend), Vercel (frontend)

Contents

EXECUTION PHASES

01	Product & Design Direction	3
02	System Implementation	4
03	Feature Expansion	5
04	Debugging & Root-Cause Fixes	6
05	LLM Provider Infrastructure (Ollama + Groq)	7
06	Augmented Reality Implementation	8

APPENDICES

A	Production System Prompt	9
B	System Architecture & Workflow Diagram	10

HOW THIS DOCUMENT IS ORGANIZED

Six execution phases

Each phase records the objective, the exact prompt issued to the AI coding agent, and the verified execution outcome — in the order the system was actually built.

Governing constraints

Real, verifiable KOHLER data only; a strict monochrome design system; and auditable, typed actions in place of free-form AI text — enforced from Phase 01 onward.

Two appendices

The exact production system prompt governing the live agent, followed by a single diagram tracing input to deployment end to end.

01

Product & Design Direction

PHASE 01 OF 06

OBJECTIVE	AI / TOOL USED	INPUT	OUTPUT
Establish product scope, real-data constraints, and visual design direction	Claude	KOHLER India website, two Color-Hex UI palettes	Product/design direction brief used as the foundation for the PRD

PROMPT

```
I am building an AI-powered KOHLER Bathroom Designer & Planner as a case-study project for a competition.

1. PRODUCT REFERENCE
Use the official KOHLER India website as the primary reference for bathroom products and categories. Only use bathroom-related KOHLER products. Do not invent products, SKUs, prices, specifications, or features. Mark unverifiable information as unavailable rather than estimating it.

2. VISUAL DESIGN DIRECTION
Premium, minimal, modern visual language. Use the two supplied Color-Hex palettes together as the primary UI reference (black, dark charcoal, grey, light grey, white). Avoid gradients, unnecessary decoration, and visual noise.

3. LAYOUT AND SPACING
Strong visual hierarchy and consistent, comfortable spacing across margins, component padding, cards, forms, navigation, and the 3D viewer. The interface should never feel cramped.

4. 3D BATHROOM VISUALIZATION
Must look realistic, maintain correct proportions, fit the supplied dimensions, avoid overlaps, respect clearances, and use real product models rather than placeholder primitives wherever available.

5. AI BATHROOM PLANNING
Accept dimensions, budget, style, and preferences, and recommend a configuration validated against real product data and spatial feasibility – never assume an AI recommendation is automatically valid.

6. FUTURE DEPLOYMENT
Structure the application to be deployment-ready from the start: centralized API calls, environment-variable-based secrets, clear frontend/backend separation, graceful API failure handling, no hardcoded development URLs.

7. ACCURACY RULE
Never fabricate or silently substitute missing product data. Clearly distinguish verified KOHLER data, unverified data, AI-generated recommendations, and placeholder content.
```

EXECUTION OUTCOME

This brief established three constraints that governed every later phase: real, verifiable product data only; a strict monochrome UI system derived from the two supplied color palettes (merged into a single grayscale token scale);

and a 3D visualization built around actual product geometry rather than generic shapes. The category taxonomy was independently verified against the live KOHLER India site before being written into the design specification.

Governing constraint carried forward: data authenticity, monochrome design system, and real-geometry 3D — every subsequent phase is checked against these three rules.

02

System
Implementation

PHASE 02 OF 06

OBJECTIVE	AI / TOOL USED	INPUT	OUTPUT
Turn the requirements into a working, milestone-ordered full-stack application	AI coding agent (Antigravity)	PRD + starter Playwright catalog scraper	Verified data pipeline, backend, optimizer, 3D scene, chatbot, UI

PROMPT

The PRD is the source of truth for product scope, features, architecture, UI, design system, 3D requirements, APIs, and deployment. Do not invent a different product structure when the PRD already specifies the required behavior. The scraper is a starting point – its selectors are not guaranteed to match the live site and must be verified.

IMPORTANT PRINCIPLES: use real KOHLER India data only; never fabricate names, SKUs, prices, or specifications; do not silently replace missing data with assumptions; preserve working functionality; do not move to the next milestone until the current one works; no large unrelated refactors.

STEP 1 – DATA

Verify and fix the product scraper against the real rendered DOM; produce kohler_catalog.json; report total/verified/unverified products and example verified entries. Do not continue if the data is unreliable.

STEP 2 – SCHEMA

Build the PostgreSQL schema; load only verified product information; distinguish verified data, missing data, derived data, and AI-generated recommendations.

STEP 3 – BACKEND

Implement the FastAPI backend (/optimize, /chat, /catalog, /export). The optimizer must use real catalog data, room dimensions, budget, theme, and clearance constraints – not hardcoded mock products.

STEP 4 – 3D VIEWER

Implement the React Three Fiber 3D visualization: correct proportions, real product models where available, clearly identified placeholders otherwise, automatic camera fitting for any room size.

STEP 5 – AGENT

Implement the LLM-based conversational agent: understands normal conversation, extracts requirements, calls catalog/optimizer tools, and is not a set of hardcoded responses pretending to be an LLM.

STEP 6 – DESIGN SYSTEM

Apply the PRD design system exactly: color tokens, typography, 8px spacing system, responsive layout.

STEP 7 – DEPLOYMENT

Environment variables, CORS, no hardcoded localhost dependencies, no exposed secrets, graceful API failure handling.

After each step: explain what changed, what was verified, what remains unverified, and any assumptions – stop and ask before making a major decision the PRD leaves ambiguous.

EXECUTION OUTCOME

The scraper was corrected after confirming that KOHLER India's category pages render their product grid client-side via JavaScript — a Playwright-based approach was required, visiting individual product pages (which do render server-side) to extract verified name, SKU, and price data. The remaining milestones were implemented in the specified order: verified catalog and database, optimization engine, 3D scene, conversational agent, and UI, each confirmed working before the next began.

03 Feature Expansion

PHASE 03 OF 06

OBJECTIVE	AI / TOOL USED	INPUT	OUTPUT
Add chatbot UI, realistic 3D products, 2D/3D toggle, vision input, and AR	AI coding agent (Antigravity)	Existing codebase and PRD	Expanded feature set across visualization, input, and export

PROMPT

Continue from the existing codebase. Do not rebuild from scratch. Implement the following in order:

1. CHATBOT UI + REAL LLM BACKING
Floating button, bottom-right, closed by default; genuine LLM with tool/function calling, not a rule-based system disguised as one.

2. REALISTIC KOHLER 3D PRODUCTS
Replace placeholder geometry with verified official models (Studio Kohler / BIMobject) matched to India catalog SKUs. Record match confidence (exact / reasonable / placeholder) per fixture – never silently substitute an unrelated product. Improve PBR materials, lighting, shadows, and camera framing.

3. 2D / 3D TOGGLE
Default to 3D; both views must read from a single shared placement dataset, not separate layout logic.

4. SPECS AS FORMAL OPTIMIZER INPUT
The optimizer must receive two explicit input groups: user inputs (dimensions, budget, theme) and catalog inputs (price, dimensions, clearance, compatibility), visible as a distinct schema.

5. PDF EXPORT ICON FIX
Black boxes appearing instead of icons; render icons as SVG/raster images, not icon-font glyphs.

6. IMAGE-BASED DIMENSION INPUT

Upload a photo or sketch; a vision-capable model estimates dimensions and layout; the result must populate the manual form for the user to review and confirm – never silently overwrite user input.

7. AR VIEW

Optional "View in AR" using WebXR where supported, isolated so unsupported devices are unaffected; hide/disable gracefully where AR cannot work.

8. SPACING AUDIT

Check the full 8px spacing system across every screen, not just new components.

After every item report: root cause, files changed, what was implemented and verified, and any remaining limitations or placeholders.

EXECUTION OUTCOME

Delivered iteratively across several rounds: the chat button and panel were added; the initial 3D render was found completely blank (a separate rendering bug), then rebuilt with real sourced product geometry, PBR materials, and HDRI lighting; the 2D/3D toggle and formal catalog-input schema were implemented; the PDF icon bug was traced to unsupported icon-font glyphs and fixed with image-based icons; image-based dimension input and AR were both brought forward from later-priority tiers into active scope at the user's request.

Scope note: items 6 and 7 (vision-based input, AR) were originally lower-priority and were escalated mid-phase at the user's request rather than deferred to a later cycle.

04 Debugging & Root-Cause Fixes

PHASE 04 OF 06

OBJECTIVE	AI / TOOL USED	INPUT	OUTPUT
Diagnose and resolve concrete functional and performance bugs	AI coding agent (Antigravity)	Existing running application	Verified fixes with evidence, not assumed fixes

PROMPT

Several concrete bugs remain. For each: reproduce the problem, trace the relevant data/control flow, identify the actual root cause, apply the smallest appropriate fix, test it, and report the result before moving to the next issue. Do not report "fixed" without explaining the underlying cause.

1. PERFORMANCE

Profile before optimizing (React DevTools, browser Performance tab); check repeated model fetching/parsing, unnecessary full-page re-renders, and duplicated API calls. Report the actual bottleneck first.

2. 3D SCENE DOES NOT UPDATE

Trace state from user action through to scene reconstruction; treat as a state/data-flow bug before assuming it is a rendering bug.

3. BLANK / MISSING 3D VIEW

Show a visible loading state while rebuilding and a visible error state on failure; never leave a blank canvas with no explanation; log which stage failed.

4. MODELS DO NOT MATCH REAL PRODUCTS

Log, per fixture, whether a real model or placeholder loaded and why; fix shape, scale, placement, finish, and remove any stray unintended geometry.

5. PRODUCT LIST CARD CLEANUP

Remove inline spec chips (Mech, pressure, flow rate) from the compact comparison list; keep them in the detail view only.

6. CHATBOT GENERAL QUERIES

A message like "2 + 2" must not return a canned "I can only help with bathroom design" response. Trace whether a classifier, narrow intent router, silent API failure, or fallback response is blocking genuine LLM replies. Tool-based requests and general conversation must both work correctly.

EXECUTION OUTCOME

Each issue was traced to a specific, verifiable root cause rather than patched superficially: the missing chat responses were traced to an unconfigured LLM API key causing a silent offline fallback mode; the 3D update failure was traced to a state-subscription gap between the tier-switch control and the 3D component; a uniform fixture color was traced to a hardcoded material value overriding real per-product finish data; and stray placeholder geometry was located and removed. Each fix was confirmed with direct evidence — logs, a rendered scene showing distinct fixture colors, a successful state update — before being marked resolved.

Diagnostic discipline applied throughout: reproduce → trace → root-cause → smallest fix → verify with evidence, repeated for all six issues before sign-off.

05 LLM Provider Infrastructure (Ollama + Groq)

PHASE 05 OF 06

OBJECTIVE	AI / TOOL USED	INPUT	OUTPUT
Support both local (Ollama) and hosted (Groq) LLM inference behind one interface	AI coding agent (Antigravity)	Existing chat/tool-calling backend	Provider-independent LLM layer, verified end to end

PROMPT

Update the LLM architecture to support both local Ollama inference and Groq API inference, with no frontend changes required when switching providers.

1. PROVIDER ABSTRACTION

A single backend LLM interface; the rest of the application should not know which provider is active.

2. CONFIGURATION

Environment variables only (LLM_PROVIDER, GROQ_API_KEY, GROQ_MODEL, OLLAMA_BASE_URL, OLLAMA_MODEL). Never expose keys in frontend code or commit .env files.

3. GROQ

Production-friendly provider; server-side calls only; handle timeouts, rate limits, and provider errors; return useful errors to the frontend without exposing the key.

4. OLLAMA

Local-development provider; configurable base URL/model; clear error when unavailable; keep the model warm where supported to reduce cold-start latency.

5. TOOL CALLING

Both providers expose the same application-level tools through a provider-independent interface.

6. FALLBACK BEHAVIOR

If the configured provider fails, return a clear error; never fabricate a response or silently swap in a hardcoded answer.

7. VERIFICATION

Test the same design request and a general conversational query ("2 + 2") against both providers; report provider used, model used, response time, tool calls, and errors. Do not claim a provider works unless it has actually been tested.

EXECUTION OUTCOME

This phase surfaced the most extensive debugging chain in the project. A missing API key was found first, followed by a global (rather than per-session) concurrency lock serializing every chat request app-wide, then a Groq model ID that had been deprecated by the provider mid-project, then a stale terminal-session environment variable shadowing the correct key during local testing, and finally a secret-masking wrapper that was passing a masked display string into the live API request instead of the real key. Each layer was isolated with direct evidence — startup logs, health-endpoint output, and a direct provider API call made outside the application — before the next hypothesis was tested, until the chatbot was confirmed genuinely functioning end to end on the Groq provider.

Root-cause chain: missing key → global concurrency lock → deprecated model ID → shadowed env variable → masked secret injected into the live call. Five distinct layers, each confirmed before moving to the next.

06 Augmented Reality Implementation

PHASE 06 OF 06

OBJECTIVE	AI / TOOL USED	INPUT	OUTPUT
Implement and debug cross-platform AR without affecting the rest of the app	AI coding agent (Antigravity)	Existing 3D scene and product placement data	WebXR (Android) + Apple AR Quick Look (iOS) experience

PROMPT

Work ONLY on AR functionality. Do not modify unrelated UI, chatbot, LLM provider, catalog, optimizer, PDF export, or 2D/3D desktop rendering. Existing working Android AR must remain working.

1. GOAL

Launch the generated design in AR on a supported device, representing the full selected suite (not one fixture) unless a single-product view was explicitly requested, preserving identity, position, scale, and rotation.

2. TRACE THE COMPLETE FLOW

Document the exact path from button click through device detection, iOS/Android branching, asset selection, and session launch, before changing any code.

3. ANDROID / WEBXR

Preserve the working experience; verify WebXR availability, camera permission, and full-suite rendering with correct positions and scale.

4. iOS / APPLE QUICK LOOK

Inspect the actual .usdz asset directly; never substitute placeholder cubes/planes to make it appear to work.

5. FULL SUITE REQUIREMENT

The AR asset must contain every fixture in the generated bundle; do not resolve a full-suite bug by pointing the button at a single-product asset.

6. POSITION AND SCALE

Correctly convert Three.js coordinates to the target AR coordinate system, accounting for unit system, axis convention, origin, and rotation.

7. ASSET CACHING

Verify the device is not loading a stale cached version of the asset after an update.

8. DEVICE DETECTION

Android uses WebXR, iOS uses Apple Quick Look, unsupported desktop/browser hides the AR action without breaking the rest of the site.

9. ERROR HANDLING

AR failure must never break the main application.

10. VALIDATION

Test Android full suite, iOS full suite, single product, and unsupported browser separately. Do not claim iOS AR is fixed unless actually tested on a real iPhone/Safari/Quick Look environment, or the limitation is explicitly stated instead.

EXECUTION OUTCOME

The Android WebXR path was confirmed working after fixing an initial launch failure. iOS AR was implemented as a separate Apple AR Quick Look (.usdz) path with platform detection choosing the correct flow per device, and the AR control was made to hide itself entirely on platforms where AR genuinely cannot run (e.g. desktop browsers), rather than surfacing a technical diagnostic to the end user. The "Launch Full Suite in AR" flow — which initially rendered only a single fixture instead of the complete bundle — was scoped for a dedicated, isolated fix following the same trace-before-modify discipline, kept deliberately separate from all other application code.

Isolation constraint: every change in this phase was scoped to AR only — chatbot, optimizer, catalog, PDF export, and desktop 2D/3D rendering were explicitly out of bounds.

A

Production System Prompt

APPENDIX A

The following system prompt governs the deployed conversational agent's behavior at runtime, distinct from the development-time instructions used to build the system.

SYSTEM PROMPT — CONVERSATIONAL DESIGN AGENT

You are the official KOHLER AI Bathroom Designer assistant for KOHLER India. Your mission is to help customers design their dream bathroom using real verified KOHLER fixtures, adhering to aesthetic themes (Minimalist Modern, Classic Luxury, Japanese Zen), space constraints, and budget targets. **CRITICAL RULES**

- 1 Whenever the user requests layout modifications, style changes, or budget adjustments, call the appropriate tool (*reoptimize* or *swap_item*).
- 2 Never invent product SKUs, prices, or layout dimensions.
- 3 Keep conversational replies concise, elegant, and focused on KOHLER luxury aesthetics.

■ EXPOSED TOOL INTERFACE

The agent is exposed exactly three tools, each with a strictly typed schema, so that every change to the active design passes through an auditable function call rather than free-form text generation. No mutation to price, layout, or product identity is ever expressed as free-form text — it is always a typed call the backend can log, validate, and roll back.

reoptimize() Re-runs the multi-constraint optimizer against updated budget, theme, or space parameters. Triggered whenever the customer changes a high-level constraint mid-conversation.	swap_item() Replaces a single fixture in the active suite with a compatible catalog-verified alternative, without disturbing the rest of the recommended bundle.	get_bundle_details() Retrieves full specification, pricing, and clearance data for the active recommended bundle, for display or quotation.
---	--	---

Design intent. Constraining the agent to three typed tools keeps every design mutation auditable — no change to price, layout, or product identity happens through unstructured text. This mirrors the accuracy rule set in Phase 01: verified data in, auditable actions out. Appendix B traces how this conversational layer fits into the full system, from initial input through catalog grounding and optimization to the deployment layer.

B

System Architecture & Workflow Diagram

APPENDIX B

The diagram traces the full path from user input through to the deployed application, organized into five layers: input, data & grounding, decisioning, client-facing outputs, and deployment. The dashed loop on the left shows the only path by which the conversational agent may mutate the active design — through the same auditable tool calls defined in Appendix A.

